

哈夫曼树创建过程交互演示说明文档

数据结构课程作业

2026 年 5 月

1 项目概述

本项目围绕“哈夫曼树创建”这一数据结构主题，制作了一个可部署到 GitHub Pages 的交互式网页。用户可以输入字符与权值，点击按钮逐步观察哈夫曼树的构建过程：初始化优先队列、取出两个最小节点、合并新节点、重新入队，直到队列中只剩根节点。构建完成后，网页会根据“左分支为 0，右分支为 1”的规则生成哈夫曼编码表。

项目最终包含三个主要成果：

- 交互式网页：index.html、styles.css、app.js。
- 说明文档源文件：report.tex。
- 说明文档 PDF：report.pdf。

部署后的访问地址规划为：

```
https://ccycyc6.github.io/ds_work/
```

2 哈夫曼树创建思路

哈夫曼树是一种带权路径长度最短的二叉树，常用于压缩编码。构建过程可以用优先队列描述：

1. 将每个字符及其权值看作一个叶子节点。
2. 将所有叶子节点按权值从小到大放入优先队列。
3. 每次从队列中取出权值最小的两个节点。
4. 创建一个新的父节点，其权值等于两个子节点权值之和。
5. 将两个节点分别作为新节点的左孩子和右孩子。

6. 把新节点重新放入优先队列。
7. 重复上述过程，直到队列中只剩一个节点，该节点就是哈夫曼树的根。

网页演示中采用稳定排序规则：如果两个节点权值相同，则按照它们进入队列的先后顺序排序。这样可以保证同一组输入每次得到的演示结果一致。

3 网页功能说明

3.1 输入与初始化

用户输入格式为：

A:5, B:9, C:12, D:13, E:16, F:45

每一项由字符、冒号和正整数权值组成，多项之间可以使用逗号或换行分隔。点击“初始化”后，程序会检查输入是否为空、字符是否重复、权值是否合法，然后生成初始优先队列。

3.2 逐步构建

点击“下一步”时，页面会依次展示以下动作：

- 高亮当前取出的两个最小节点。
- 显示两个节点合并后的新父节点。
- 更新优先队列。
- 在 SVG 区域绘制当前树结构。
- 在步骤说明中记录本次操作。

3.3 编码生成

当队列中只剩一个根节点时，构建结束。程序从根节点开始遍历整棵树，左边路径记为 0，右边路径记为 1，最终得到每个叶子字符的哈夫曼编码。

4 关键实现

4.1 算法设计

核心逻辑位于 `app.js`。程序先将输入解析为叶子节点数组，然后通过 `buildSteps` 函数预先生成所有演示步骤。每个步骤都保存当前队列、当前树根、被选中的节点以及步骤说明。这样页面渲染时只需要根据当前步骤更新视图，交互逻辑比较清晰。

核心伪代码如下：

```
queue = sort(leaves)
while queue.length > 1:
    first = queue.popMin()
    second = queue.popMin()
    parent = new Node(first.weight + second.weight)
    parent.left = first
    parent.right = second
    queue.push(parent)
    queue = sort(queue)
root = queue[0]
```

4.2 可视化设计

树形结构使用 SVG 绘制。程序递归遍历当前根节点，为每个叶子节点分配横向位置，再让父节点位于左右子树的中间。边上标注 0 或 1，节点中显示字符或内部节点编号以及权值。取出的节点、合并节点和叶子节点使用不同颜色区分。

4.3 页面结构

页面分为五个区域：

- 输入与控制区。
- 优先队列区。
- 树形动画区。
- 步骤说明区。
- 哈夫曼编码表区。

网页采用原生 HTML、CSS 和 JavaScript 实现，不依赖框架和构建工具，因此可以直接部署到 GitHub Pages。

5 AI 工具使用说明

本项目使用 AI 辅助完成需求拆解、交互设计、代码生成、文档撰写和测试修正。AI 的作用不是替代算法理解，而是帮助把课程要求转化为可运行的网页和规范化说明文档。

5.1 工具介绍

工具	作用
ChatGPT / Codex	分析作业要求，制定实现计划，生成网页代码、JavaScript 算法逻辑、CSS 样式和 LaTeX 文档。
终端命令	检查目录、运行 Node.js 校验算法、编译 LaTeX、启动本地静态服务器。
XeLaTeX	将中文 LaTeX 源文件编译为 PDF。
GitHub Pages	托管静态网页，提供可访问的 Web 链接。

5.2 提示词记录

以下记录按实际完成项目的阶段整理，保留了关键提示词和 AI 输出目标。

第 1 轮：提出作业需求

用户提示词：

设计并制作一个交互式网页或演示动画，直观地展示某种树形数据结构的构建或操作过程，我要做哈夫曼树创建，做成一个可访问的 Web 链接，然后说明文档(PDF)（用`latex`）：关键步骤说明：整体思路阐述，解释如何利用 AI 工具分步完成上述动画。
工具介绍、提示词记录：完整、详细记录你与 AI 的交互过程。

AI 输出目标：确认作业需要同时包含网页动画、Web 链接、PDF 文档和 AI 交互记录。

第 2 轮：确认交付方式

AI 提问要点：

你希望最终的“可访问 Web 链接”是哪一种？
哈夫曼树创建动画希望做到哪种交互深度？
PDF 说明文档使用什么语言和记录粒度？

用户选择：

GitHub Pages
逐步构建
中文详细

AI 输出目标：将项目定位为可部署到 GitHub Pages 的静态站点，并确定交互方式为逐步构建。

第 3 轮：形成实施计划

AI 生成计划摘要：

创建 `index.html`、`styles.css`、`app.js`、`report.tex`、`report.pdf` 和 `README.md`。
网页支持输入权值、初始化、下一步、重置、示例数据。
算法按优先队列构建哈夫曼树，完成后生成编码表。
GitHub Pages 访问地址为 https://ccycyc6.github.io/ds_work/。

AI 输出目标：给出可执行的文件结构、功能范围、测试方式和部署方式。

第 4 轮：开始实施

用户提示词：

PLEASE IMPLEMENT THIS PLAN

AI 执行目标：

- 编写静态网页文件。
- 实现哈夫曼树算法和逐步动画。
- 编写中文 LaTeX 说明文档。
- 编译 PDF。
- 初始化 Git 仓库并连接到 GitHub 仓库。

5.3 AI 分步完成动画的方法

AI 完成动画时采用了分层拆解的方式：

1. 先把哈夫曼树创建算法抽象为“状态序列”，每一个状态代表一个演示步骤。
2. 再设计页面区域，让输入、队列、树形图、说明文字和编码表分别承担不同的信息展示任务。
3. 然后编写纯 JavaScript 算法函数，保证不依赖页面也能校验构建结果。
4. 接着使用 SVG 根据当前树节点递归计算坐标并绘制节点和连线。
5. 最后补充输入校验、按钮状态、错误提示和响应式 CSS。

6 运行与部署

6.1 本地运行

在项目目录运行：

```
python3 -m http.server 8000
```

然后在浏览器打开：

```
http://localhost:8000/
```

6.2 编译 PDF

使用 XeLaTeX 编译：

```
xelatex report.tex  
xelatex report.tex
```

6.3 部署到 GitHub Pages

将项目推送到仓库：

```
git init  
git branch -M main  
git remote add origin https://github.com/ccycyc6/ds_work  
git add .  
git commit -m "Create Huffman tree interactive demo"  
git push -u origin main
```

然后在 GitHub 仓库中进入 Settings -> Pages，选择 Deploy from a branch，分支选择 main，目录选择 /root。保存后访问：

```
https://ccycyc6.github.io/ds_work/
```

7 测试记录

测试用例包括：

- 示例输入 A:5, B:9, C:12, D:13, E:16, F:45 可以完整构建出哈夫曼树。
- 空输入会提示至少需要两个字符权值项。
- 重复字符会提示字符重复。

- 非数字、零或负权值会提示权值必须为正整数。
- 相同权值按输入顺序稳定排序，保证演示结果可复现。

8 总结

本项目将哈夫曼树的抽象构建过程转化为可观察、可交互的网页动画。通过逐步按钮，用户可以清楚看到优先队列如何变化、节点如何合并、树如何增长，以及最终编码如何产生。AI 工具主要用于辅助需求拆解、代码实现、文档组织和测试修正，最终形成了可部署、可演示、可提交的课程作业成果。